

Modül 13: Lambda'lar

Bir lambda, tam olarak bir soyut metodu olan bir arayüzün, sınıf töreni olmadan yazılmış uygulamasıdır.

Bu tanımlı aklınızda tutmaya değer. Lambda, Java'da yeni bir değer türü değildir. Hâlâ bir arayüz uygulayan bir nesnedir. Modül 07, tek soyut metotlu bir arayüzün sınıf yazmadan uygulanabileceğini söylemişti; bu modül o cümlelerin uygulamaya dökülmüş hâli.

Neler Öğreneceksiniz

- Lambda sözdizimi ve derleyicinin sizin için neyi çıkardığı
- Dört metot referansı biçimi, birebir aynı görünen ikisi dahil
- Yakalama ve yakalanan yerelin neden etkin final olması gerektiği
- Standart fonksiyonel arayüzler ve nasıl birleştikleri
- `java.util.function` paketinde neden bu kadar çok neredeyse aynı tip olduğu

Başka Bir Dilden Geliyorsanız

	Python / JS	Java
Anonim fonksiyon	<code>lambda</code> , <code>=></code>	<code>-></code>
Çok ifadeli gövde	JS evet, Python hayır	evet, süslü parantezle
Yakalanan değişken değişebilir	evet	hayır , etkin final olmalı
Lambda'nın tipi	fonksiyon nesnesi	arayüz uygulaması
Fonksiyon tipi	birinci sınıf	yok; bir arayüz kullanın

Önemli olan iki satır: Java'nın **fonksiyon tipi yoktur**, dolayısıyla her lambda bir hedef arayüze ihtiyaç duyar, ve yakalanan bir yerel **yeniden atanamaz**.

Ders

Aynı şey, üç biçimde

`Lambdas.java` dosyasından:

```
Greeter named = new PoliteGreeter();           // a named class
Greeter anonymous = new Greeter() {             // an anonymous class
    @Override public String greet(String n) { return "Good day, " + n; }
};
Greeter lambda = name -> "Good day, " + name;    // a lambda
```

Üçü de `Greeter` uygulayan bir nesne üretir. Lambda; arayüz adını, metod adını, niteleyicileri ve parametre tipini atlar, çünkü derleyici hepsini hedef tipten çıkarabilir.

Sözdizimi biçimleri:

```
() -> doSomething()           // no parameters, parentheses required
name -> "hello " + name        // one parameter, parentheses optional
(String name) -> ...           // explicit type when inference needs help
name -> { ...; return x; }      // block body needs braces and return
(a, b) -> a + b                // two parameters always need parentheses
```

Fonksiyonel arayüzün tam olarak bir soyut metodu vardır. Varsayılan ve statik metodlar sayılmaz, `Comparator` sınıfının onlarca metodu olup yine de lambda hedefi olabilmesinin sebebi bu. Kendi arayüzlerinize `@FunctionalInterface` yazın: derleyicinin kuralı zorlamasını sağlar, böylece sonradan ikinci bir soyut metod eklemek, onu kullanan her lambda'da değil arayüzün kendisinde başarısız olur.

Metot referansları

Dört biçim, `MethodReferences.java` dosyasından:

Biçim	Örnek	Eşdeğer lambda
statik	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
kurucu	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>
bağlı örnek	<code>prefix::concat</code>	<code>s -> prefix.concat(s)</code>
bağımsız örnek	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>

Bağımsız biçim insanların yanlış okuduğu biçimdir. Lambda'nın ilk argümanı **alıcı** olur, argüman değil.

`toUpperCase` hiç parametre almamasına rağmen `Function<String, String> upper = String::toUpperCase` yazımının çalışmasının sebebi budur.

Üç ve dördüncü biçimler aynı görünür ama değildir:

```
bound, arg "fix":           pre-fix
bound, arg "amble":        pre-amble
unbound, ("pre-", "fix"):   pre-fix
unbound, ("post-", "fix"):  post-fix
```

Bağlı olan her zaman `prefix` üzerine ekler. Bağımsız olan alıcısını ilk argümandan alır, dolayısıyla üzerine eklediği şey çağrıdan çağrıya değişir. `::` işaretinin solunda ne olduğuna bakarak ayırt edin: değişken bağlı, tip adı bağımsız ya da statik demektir.

Ne zaman kullanılmaz. `words.forEach(System.out::println)` daha nettir. Ama başka bir şey olduğu anda lambda daha iyi okunur. Bir metod referansı sığsın diye kodu eğip bükmek genellikle onu kötüleştirir.

Yakalama

Yakalanan bir yerel **final** ya da **etkin final** olmalıdır.

Etkin final, ilklendirmeden sonra ona hiç atama yapmadığınız anlamına gelir. Anahtar kelimeyi yazmazsınız; derleyici davranışı denetler. `NotEffectivelyFinal.java`:

```
error: local variables referenced from a lambda expression must be final or effectively final
```

Kuralın var olma sebebi: bir lambda, onu oluşturan metottan daha uzun yaşayabilir. Çalıştığında o yığın çerçevesi çoktan gitmiş olabilir, dolayısıyla lambda değişkene referans değil **değerin bir kopyasını** tutar. Yeniden atamaya izin verilseydi iki sorunun iyi cevabı olmazdı: zaten oluşturulmuş bir lambda yeni değeri görür mü, ve iki iş parçacığı aynı anda atarsa ne olur?

Kural değişkenle ilgilidir, nesneyle değil. Bu sorun değil:

```
List<String> items = new ArrayList<>();
Runnable r = () -> items.add("x");           // the variable never changes
```

İnsanların kuralı aşmak için kullandığı boşluk da bu:

```
int[] counter = {0};
list.forEach(x -> counter[0]++);           // compiles, and is a smell
```

Derlenir çünkü değişken hiç değişmez. Kötü kokar çünkü lambda mutasyon için kullanılıyor, ki Modül 14'ün var olma sebebi tam olarak onu değiştirmek.

Alanlarda böyle bir kural yoktur. Bir lambda `this` yakalar ve alanı onun üzerinden okur, dolayısıyla değiştirilebilir bir alan çalışır. Bu ayrıca saklanan bir lambda'nın tüm kuşatan nesneyi hayatta tuttuğu anlamına gelir, ki bu uzun ömürlü dinleyici listelerinde önemlidir.

Lambda ile anonim sınıf arasında `this` farklıdır:

```
inside a lambda, this is:           Capture
inside an anonymous class, this is: Capture$1
```

`$1` gerçek, ayrı bir sınıftır. Lambda böyle bir şey oluşturmaz, dolayısıyla içindeki `this` kuşatan nesneyi ifade eder. Bu bir davranış farkıdır, stil tercihi değil.

Standart arayüzler

Arayüz	Biçim	Metot
<code>Function<T,R></code>	T girer, R çıkar	<code>apply</code>
<code>Predicate<T></code>	T girer, boolean çıkar	<code>test</code>
<code>Supplier<T></code>	hiçbir şey girmez, T çıkar	<code>get</code>
<code>Consumer<T></code>	T girer, hiçbir şey çıkmaz	<code>accept</code>

Artı `BiFunction`, `BiPredicate`, `BiConsumer`, `UnaryOperator<T>` (bir `Function<T,T>`) ve `BinaryOperator<T>`.

Birleştirme, `StandardInterfaces.java` dosyasından:

```
twice.andThen(plusOne) on 5: 11
twice.compose(plusOne) on 5: 12
```

`andThen` "önce beni, sonra argümanı yap" demektir. `compose` "önce argümanı, sonra beni yap" demektir. İşlemler değişmeli olmadığında bunları ters çevirmek klasik bir hatadır.

`Predicate`'ler `and`, `or` ve `negate` ile birleşir.

İlkel tip özelleştirmeleri var çünkü `Function<Integer,Integer>` her değeri kutular. `IntPredicate`, `ToIntFunction`, `IntUnaryOperator` ve `long` ile `double` eşdeğerleri bundan kaçınır. Bu, Modül 02'nin kutulama bedelinin API tasarımında görünmesidir, ve `java.util.function` paketinin bu kadar büyük olmasının sebebi.

Çalıştırın

```
java modules/13-lambdas/examples/Lambdas.java
java modules/13-lambdas/examples/MethodReferences.java
java modules/13-lambdas/examples/Capture.java
java modules/13-lambdas/examples/StandardInterfaces.java

# Bilerek başarısız olur.
java modules/13-lambdas/examples/NotEffectivelyFinal.java

./scripts/verify-examples.sh modules/13-lambdas
```

Sık Yapılan Hatalar

Yakalanan bir yereli yeniden atamaya çalışmak. Önce nihai değeri hesaplayın ya da ikinci bir değişken kullanın.

Biriktirmek için `int[] box = {0}` **numarası**. Derlenir. Ayrıca lambda'nın bir indirgeme olmak istediğini söyler, ki Modül 14 onu anlatıyor.

andThen ile **compose** **kariřtırmak**. Sesli okuyun: "and then" ikinci, "compose" etrafını sarar.

Karřılařtırıcı olarak `a.length() - b.length()` **yazmak**. Büyük deęerlerde taşar.

`Comparator.comparingInt` kullanın.

Lambda içindeki `this` **deęerinin lambda olmasını beklemek**. Atıfta bulunacak bir lambda nesnesi yoktur.

Standart biri uyarken kendi fonksiyonel arayüzünüzü bildirmek. `Function<String,Integer>` bilen bir okuyucu, gidip sizin `StringScorer` arayüzünüze bakmak zorunda kalır.

Lambda'ları `this` etkisini düşünmeden uzun ömürlü koleksiyonlarda saklamak. Lambda kuřatan nesneyi tutar, dolayısıyla o nesne toplanamaz.

Özet Çıkarımlar

- **Lambda, tek soyut metotlu bir arayüzü uygular**. Java'nın fonksiyon tipi yoktur.
- `@FunctionalInterface` derleyicinin kuralı zorlamasını sağlar, sahip olduęunuz arayüzlerde.
- **Dört metot referansı biçimi**, ve baęsız olan ilk argümanı alıcıya çevirir.
- **Yakalanan yereller etkin final olmalıdır**, çünkü lambda deęeri kopyalar ve çerçeveden uzun yaşayabilir.
- **Kural deęiřkenle ilgilidir, nesneyle deęil**, dolayısıyla yakalanan bir koleksiyonu deęiřtirmek yasaldır.
- `andThen` ve `compose` **ters sırada çalışır**.
- **İlkel tip özelleřtirmeleri kutulamadan kaçınmak için vardır**.

Ödev

[homework/README.md](#)

Birleřtirilmiř predicate ve fonksiyonlardan küçük bir kural motoru kurun, sonra bilerek yakalama kuralına çarpın. Referans çözüm [solutions/13-lambdas/](#) altında.